

WEEK-9-Practical

2520080033

TASK-1

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/stat.h>

#define BUFFER_SIZE 4096

void low_level_copy(const char *src, const char *dest)
{
    int fd_src, fd_dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    fd_src = open(src, O_RDONLY);
    if (fd_src < 0)
    {
        perror("Error opening source file");
        exit(1);
    }

    fd_dest = open(dest, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_dest < 0)
    {
        perror("Error opening destination file");
        close(fd_src);
        exit(1);
    }

    while ((bytes_read = read(fd_src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(fd_dest, buffer, bytes_read);

        if (bytes_written != bytes_read)
        {
            perror("Error writing file");
            close(fd_src);
        }
    }

    printf("FILE COPY PERFORMANCE COMPARISON\n");
    printf("-----\n");

    /* Low-level I/O */
    start = clock();

    low_level_copy(source, low_dest);

    end = clock();

    low_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Low-level I/O copy completed.\n");
    printf("Time: %f seconds\n\n", low_time);

    /* Standard Library I/O */
    start = clock();

    standard_copy(source, std_dest);

    end = clock();

    std_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Standard library copy completed.\n");
    printf("Time: %f seconds\n\n", std_time);

    printf("PERFORMANCE COMPARISON\n");
    printf("-----\n");

    if (low_time < std_time)
        printf("Low-level I/O was faster.\n");
    else if (std_time < low_time)
        printf("Standard library I/O was faster.\n");
    else
        printf("Both methods took approximately the same time.\n");

    return 0;
}
```

```

root@Shamikahasini:~# nano filecopy.c
root@Shamikahasini:~# gcc filecopy.c -o filecopy
root@Shamikahasini:~# ./filecopy
FILE COPY PERFORMANCE COMPARISON
-----
Low-level I/O copy completed.
Time: 0.124761 seconds

Standard library copy completed.
Time: 0.126186 seconds

PERFORMANCE COMPARISON
-----
Low-level I/O was faster.
root@Shamikahasini:~# echo "This is a test file for Linux file copy system calls." > input.txt
root@Shamikahasini:~# yes "Linux File I/O Performance Testing" | head -c 100M > input.txt
root@Shamikahasini:~# gcc filecopy.c -o filecopy
root@Shamikahasini:~# ./filecopy
FILE COPY PERFORMANCE COMPARISON
-----
Low-level I/O copy completed.
Time: 0.093827 seconds

Standard library copy completed.
Time: 0.190291 seconds

PERFORMANCE COMPARISON
-----
Low-level I/O was faster.
root@Shamikahasini:~# ls -lh input.txt low_level_copy.txt standard_copy.txt
-rw-r--r-- 1 root root 100M Oct  2 16:09 input.txt
-rw-r--r-- 1 root root 100M Oct  2 16:09 low_level_copy.txt
-rw-r--r-- 1 root root 100M Oct  2 16:09 standard_copy.txt
root@Shamikahasini:~# cmp input.txt low_level_copy.txt
cmp input.txt standard_copy.txt
root@Shamikahasini:~#

```

Observation:

The file was successfully copied using both low-level system calls and standard library functions.

The low-level method used `open()`, `read()`, `write()`, `lseek()`, and `close()` for direct file handling.

The standard method used `fopen()`, `fread()`, `fwrite()`, and `fclose()` for buffered file operations.

The execution times were measured, showing that performance can vary depending on file size and system conditions.

Task-2

Develop a program that redirects standard input and Output using file descriptors and dup2(). Demonstrate how shells implement 1/0 redirection.

```
GNU nano 7.2 redirection.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int input_fd, output_fd;

    /* Open input file */
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("Error opening input.txt");
        return 1;
    }

    /* Open output file */
    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0)
    {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }

    /* Redirect standard input to input.txt */
    if (dup2(input_fd, STDIN_FILENO) < 0)
    {
        perror("dup2 input failed");
        return 1;
    }

    /* Redirect standard output to output.txt */
    if (dup2(output_fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 output failed");
        return 1;
    }

    /* Open output file */
    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0)
    {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }

    /* Redirect standard input to input.txt */
    if (dup2(input_fd, STDIN_FILENO) < 0)
    {
        perror("dup2 input failed");
        return 1;
    }

    /* Redirect standard output to output.txt */
    if (dup2(output_fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 output failed");
        return 1;
    }

    close(input_fd);
    close(output_fd);

    /* These now use the redirected input/output */
    char buffer[100];

    while (fgets(buffer, sizeof(buffer), stdin) != NULL)
    {
        printf("Read: %s", buffer);
    }

    return 0;
}
```

```
root@Shamikahasini:~# nano redirection.c
root@Shamikahasini:~# echo "Hello Linux I/O Redirection" > input.txt
echo "This is an example of dup2." >> input.txt
root@Shamikahasini:~# cat input.txt
Hello Linux I/O Redirection
This is an example of dup2.
root@Shamikahasini:~# gcc redirection.c -o redirection
root@Shamikahasini:~# ./redirection
root@Shamikahasini:~# cat output.txt
Read: Hello Linux I/O Redirection
Read: This is an example of dup2.
root@Shamikahasini:~#
```

Observation:

The program successfully redirected standard input and output using `dup2()` and file descriptors. The input was read from `input.txt` instead of the keyboard, while the program's output was written to `output.txt` instead of the terminal. This demonstrates how shells implement `<` and `>` redirection by changing the standard file descriptors before executing a command.